Field Suggestion Analytics - YTD Summary & Insights

```
pip install openpyxl
```

```
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting et-xmlfile (from openpyxl)
  Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
Note: you may need to restart the kernel to use updated packages.
```

```python
import pandas as pd

# Load file
file_path = "Suggestions YTD 6-11.xlsx"
df = pd.read_excel(file_path, sheet_name="in")
```

```python
df.head(5)
```

	Record Type	Posted Date	Expiration Date	Actioned	Dismissed	Marked As Complete	Title	Reason	Last Modified Date	Last Modified By: Full Name	Owner: Full Name	Accou Name
0	Schedule Call	2025-04-06	2025-04-12	1.0	NaN	NaN	High CGM Growth	<ul><li>Adriana Cadena-Martinez's CGM TRx has ...	2025-04-13	Aktana Integration	Abigail Rosales	Adrian Caden Martin
1	Schedule Call	2025-02-23	2025-03-01	1.0	NaN	NaN	Leading FSL TRx Growth with respect to Goal	<ul><li>Adriana Cadena-Martinez's FSL TRx are ...	2025-03-02	Aktana Integration	Abigail Rosales	Adrian Caden Martin
2	Schedule Call	2025-02-02	2025-02-08	1.0	NaN	NaN	HCP Dexcom Adoption for Conventional Insulin (...	<ul><li>Adriana Cadena-Martinez?s FSL TRx Shar...	2025-02-09	Aktana Integration	Abigail Rosales	Adrian Caden Martin
3	Schedule Call	2025-05-25	2025-05-31	1.0	NaN	NaN	Lagging Product Transition	<ul><li>Alberto Almeida's FSL 2+ and 3+ TRx vo...	2025-06-01	Aktana Integration	Abigail Rosales	Alberto Almeid
4	Schedule Call	2025-01-12	2025-01-18	1.0	NaN	NaN	HCP scripts are not being filled	<ul><li>Alberto Almeida's new patients have a ...	2025-01-19	Aktana Integration	Abigail Rosales	Alberto Almeid

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 154702 entries, 0 to 154701
Data columns (total 12 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   Record Type                  154702 non-null  object
 1   Posted Date                  154702 non-null  datetime64[ns]
 2   Expiration Date              154702 non-null  datetime64[ns]
 3   Actioned                     99983 non-null   float64
 4   Dismissed                    27143 non-null   float64
 5   Marked As Complete           7987 non-null    float64
 6   Title                        154702 non-null  object
 7   Reason                       154702 non-null  object
 8   Last Modified Date           154702 non-null  datetime64[ns]
 9   Last Modified By: Full Name  154702 non-null  object
 10  Owner: Full Name             154702 non-null  object
 11  Account: Name                154702 non-null  object
```

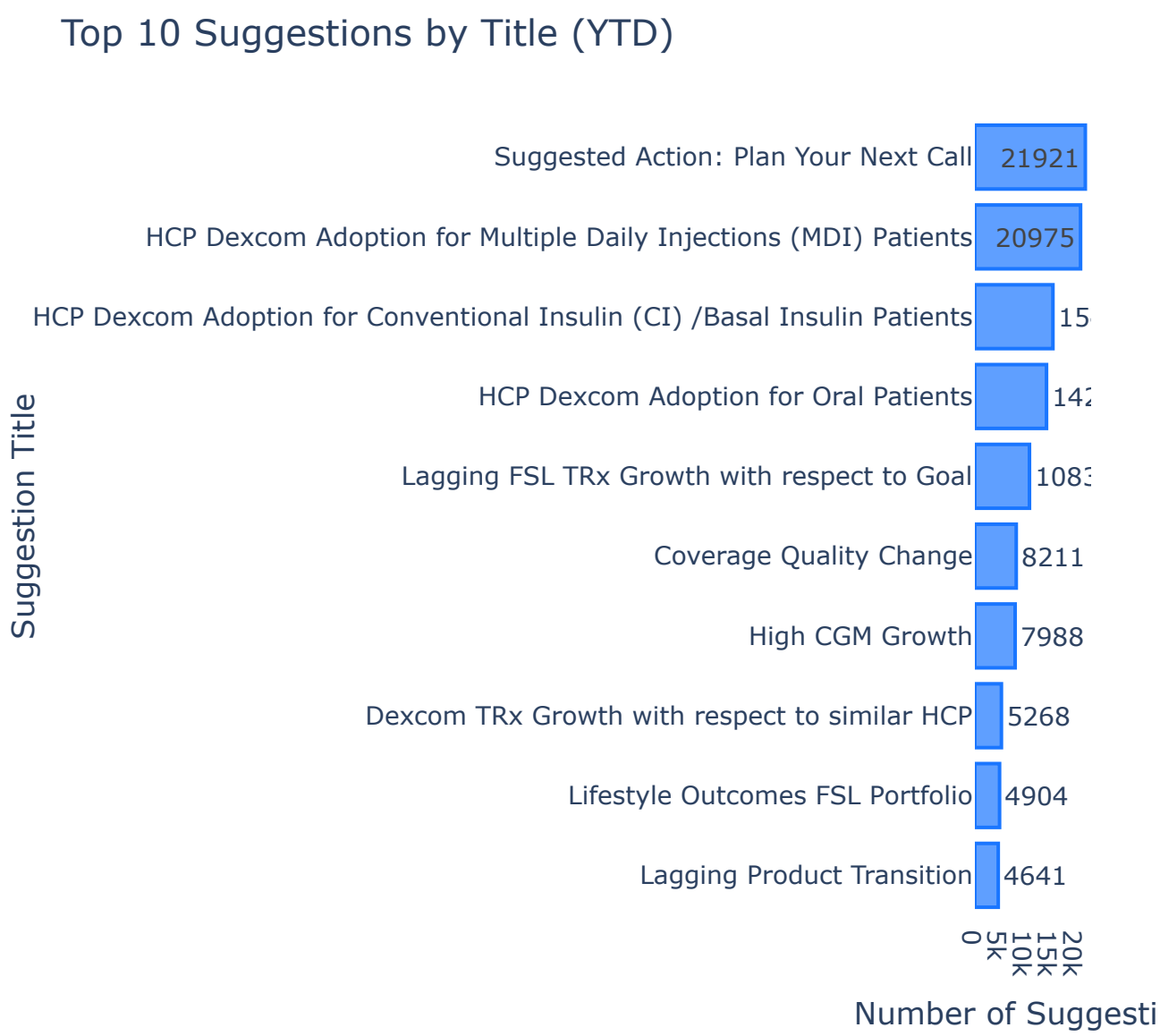
```
dtypes: datetime64[ns](3), float64(3), object(6)
memory usage: 16.2+ MB
```

```python
# Clean columns
df.columns = df.columns.str.strip()
df['Title'] = df['Title'].astype(str).str.strip()
df['Dismissed'] = df['Dismissed'].astype(str).str.lower()
df['Marked As Complete'] = df['Marked As Complete'].astype(str).str.lower()
df['Actioned'] = df['Actioned'].astype(str).str.lower()
```

```python
import plotly.express as px
import plotly.graph_objects as go
```

Visualizations

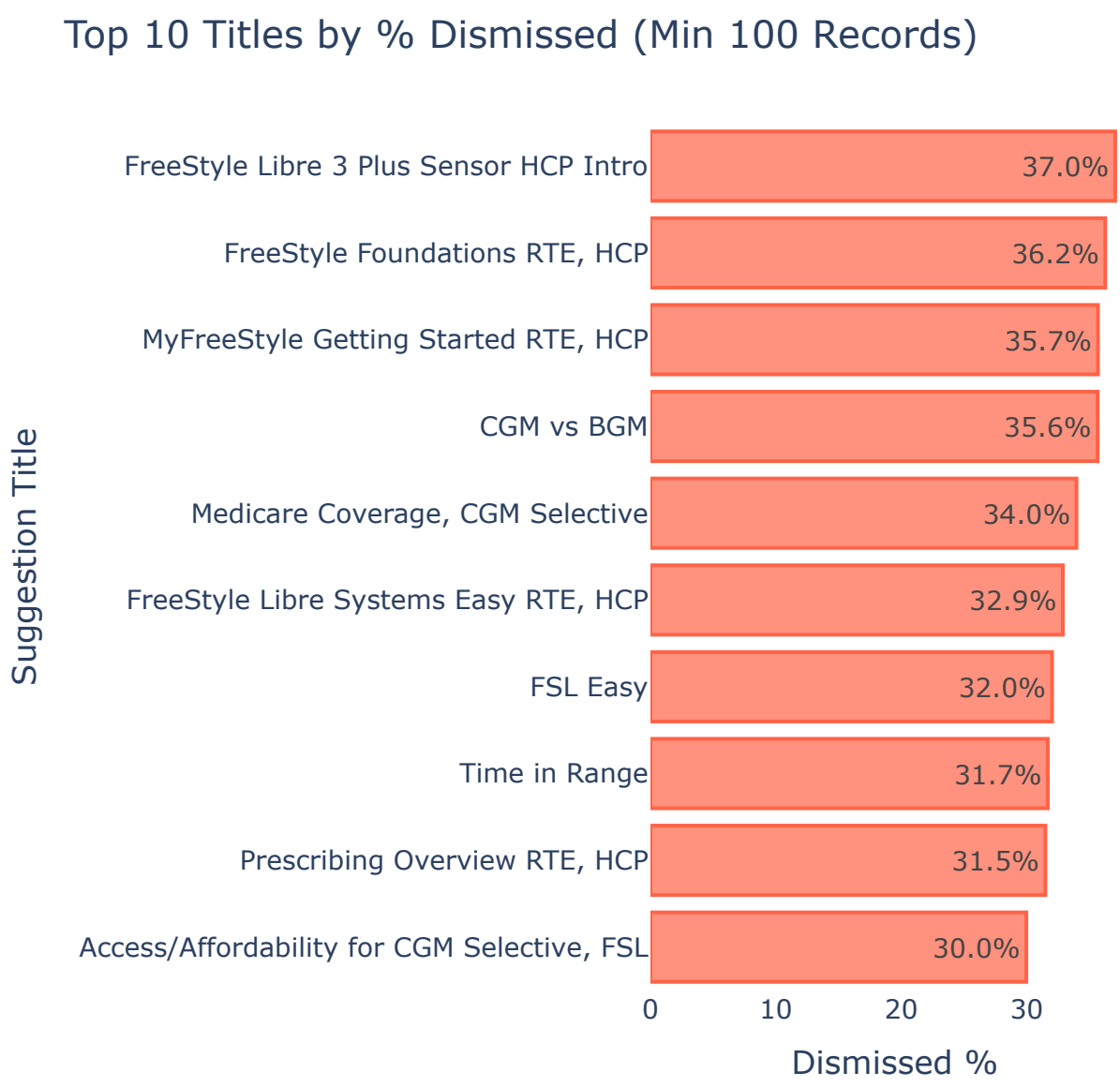
```python
# 1. Top 10 Suggestions by Title
# Sort data in ascending order for horizontal bar chart
top_titles_sorted = top_titles.sort_values()

# Create fancier bar chart
fig1 = go.Figure()

fig1.add_trace(go.Bar(
    x=top_titles_sorted.values,
    y=top_titles_sorted.index,
    orientation='h',
    marker=dict(color='rgba(26, 118, 255, 0.7)',
                line=dict(color='rgba(26, 118, 255, 1.0)', width=2)),
    text=top_titles_sorted.values,
    textposition='auto',
    hoverinfo='x+y'
))

fig1.update_layout(
    title='Top 10 Suggestions by Title (YTD)',
    xaxis_title='Number of Suggestions',
    yaxis_title='Suggestion Title',
    font=dict(size=14),
    height=600,
    plot_bgcolor='white',
    margin=dict(l=200, r=40, t=80, b=40)
)

fig1.show()
```




```
print(df['Dismissed'].unique())
['nan' '1.0']
```

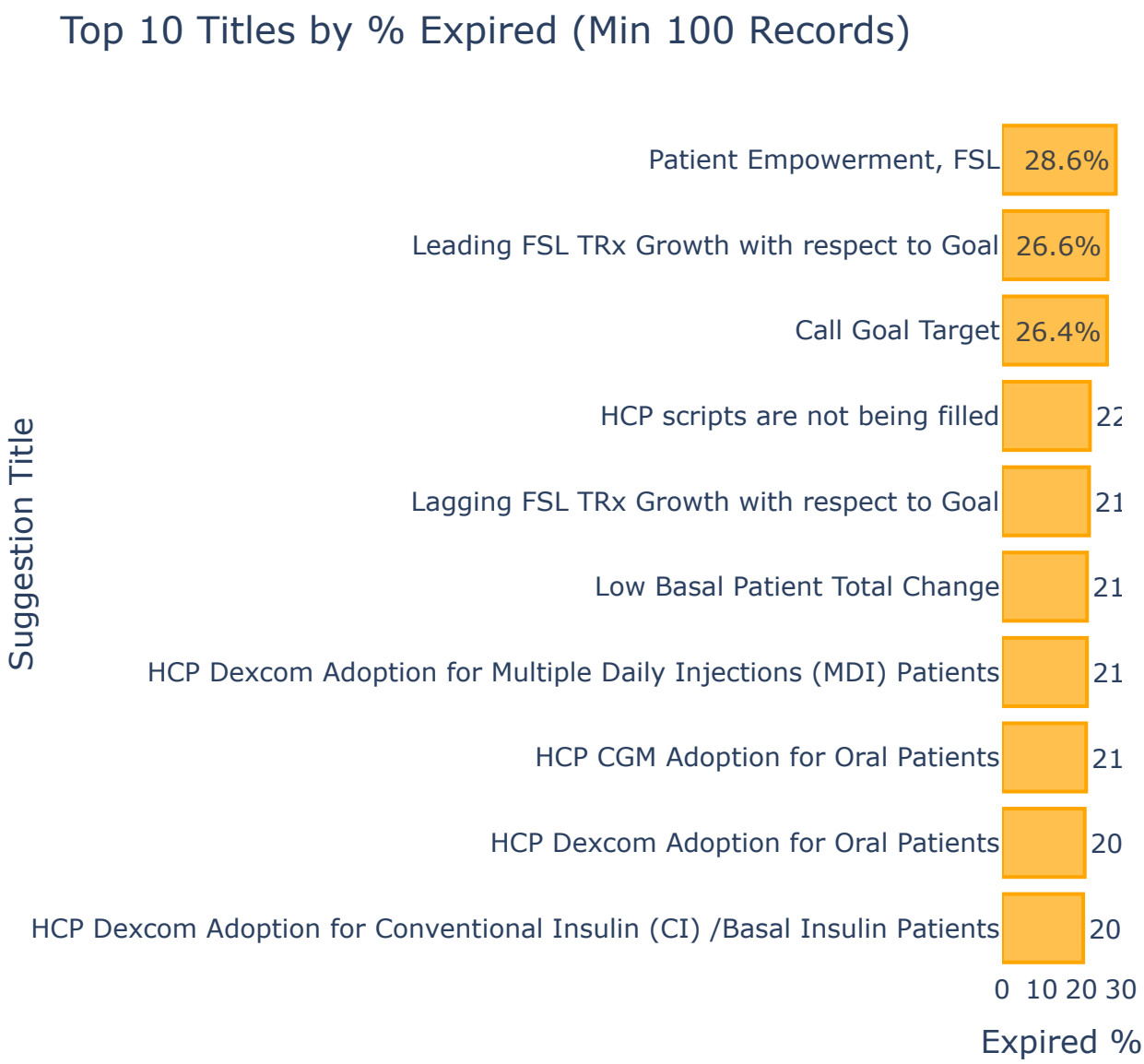
```
# 2. Top 10 by % Dismissed
# Compute % dismissed (confirmed fix for string '1.0')
dismissed_group = df.groupby('Title').agg(
    total=('Title', 'count'),
    dismissed=('Dismissed', lambda x: (x == '1.0').sum())
)
dismissed_group['dismissed_pct'] = 100 * dismissed_group['dismissed'] / dismissed_group
['total']

# Filter and sort
dismissed_filtered = dismissed_group[dismissed_group['total'] >= 100]
top_dismissed = dismissed_filtered.sort_values('dismissed_pct', ascending=False).head(1
0)
top_dismissed_sorted = top_dismissed.sort_values('dismissed_pct')

# Fancy Plot
```

```
fig2 = go.Figure()

fig2.add_trace(go.Bar(
    x=top_dismissed_sorted['dismissed_pct'],
    y=top_dismissed_sorted.index,
    orientation='h',
    marker=dict(color='rgba(255, 99, 71, 0.7)',
                line=dict(color='rgba(255, 99, 71, 1.0)', width=2)),
    text=top_dismissed_sorted['dismissed_pct'].round(1).astype(str) + '%',
    textposition='auto',
    hoverinfo='x+y'
))

fig2.update_layout(
    title='Top 10 Titles by % Dismissed (Min 100 Records)',
    xaxis_title='Dismissed %',
    yaxis_title='Suggestion Title',
    font=dict(size=14),
    height=600,
    plot_bgcolor='white',
    margin=dict(l=200, r=40, t=80, b=40)
)

fig2.show()
```



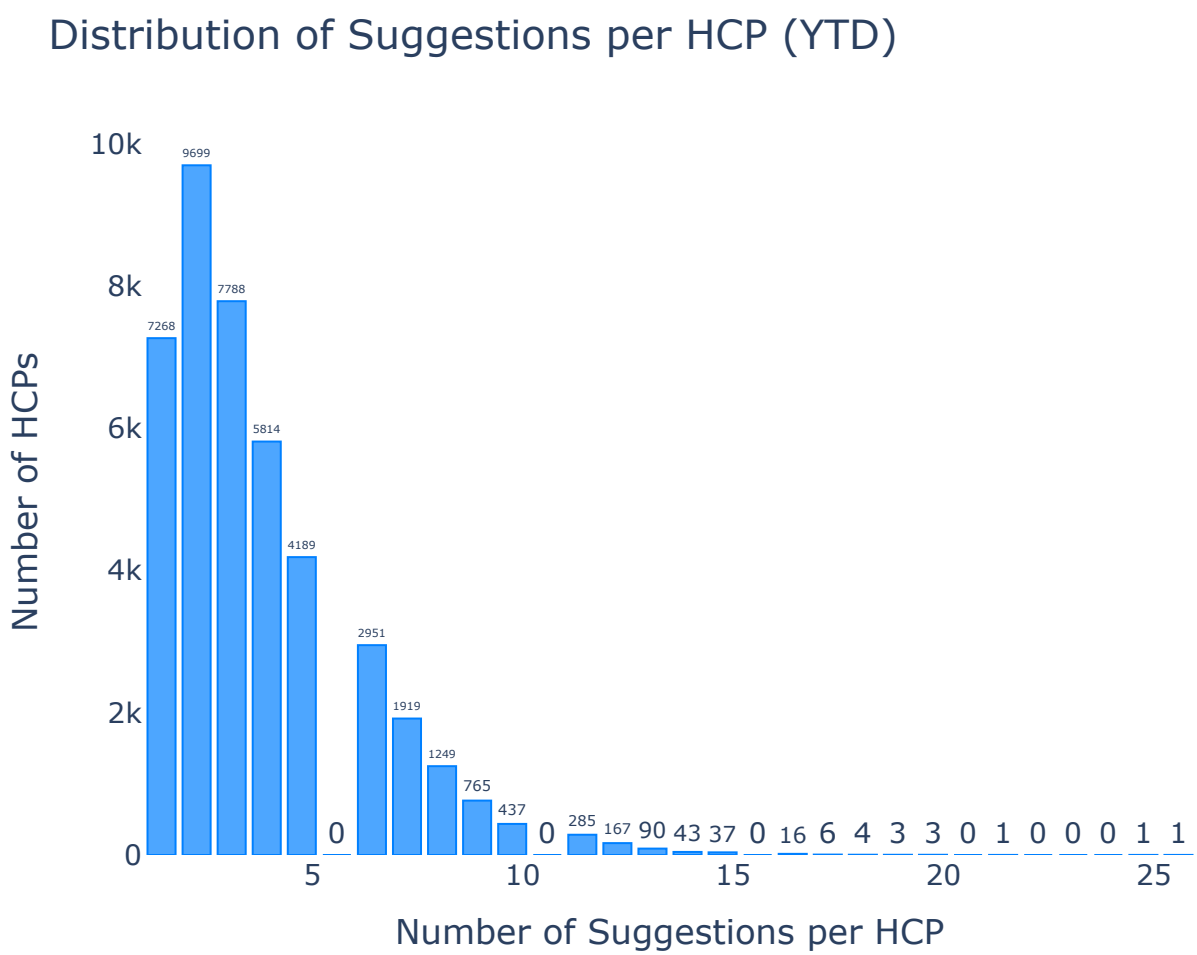

```
# 3. Top 10 by % Expired

# Define expired rows where none of the fields are '1.0'
expired_df = df[
    (df['Dismissed'] != '1.0') &
    (df['Marked As Complete'] != '1.0') &
    (df['Actioned'] != '1.0')
]

# Aggregate counts
expired_group = expired_df.groupby('Title').size().rename('expired_count')
total_group = df.groupby('Title').size().rename('total_count')

# Combine and calculate %
combined = pd.concat([expired_group, total_group], axis=1).dropna()
combined = combined[combined['total_count'] >= 100]
combined['expired_pct'] = 100 * combined['expired_count'] / combined['total_count']
top_expired = combined.sort_values('expired_pct', ascending=False).head(10)
top_expired_sorted = top_expired.sort_values('expired_pct')
```

```python
# Plot
fig3 = go.Figure()

fig3.add_trace(go.Bar(
    x=top_expired_sorted['expired_pct'],
    y=top_expired_sorted.index,
    orientation='h',
    marker=dict(color='rgba(255, 165, 0, 0.7)',  # orange tone
                line=dict(color='rgba(255, 165, 0, 1.0)', width=2)),
    text=top_expired_sorted['expired_pct'].round(1).astype(str) + '%',
    textposition='auto',
    hoverinfo='x+y'
))

fig3.update_layout(
    title='Top 10 Titles by % Expired (Min 100 Records)',
    xaxis_title='Expired %',
    yaxis_title='Suggestion Title',
    font=dict(size=14),
    height=600,
    plot_bgcolor='white',
    margin=dict(l=200, r=40, t=80, b=40)
)

fig3.show()
# Get count of suggestions per HCP
hcp_counts = df['Account: Name'].value_counts()

# Fancy histogram plot
fig4 = go.Figure()

fig4.add_trace(go.Histogram(
    x=hcp_counts,
    nbinsx=30,
    marker=dict(color='rgba(0, 128, 255, 0.7)',
                line=dict(color='rgba(0, 128, 255, 1.0)', width=1)),
    hoverinfo='x+y'
))

fig4.update_layout(
    title='📊 Distribution of Suggestions per HCP (YTD)',
    xaxis_title='Number of Suggestions per HCP',
    yaxis_title='Number of HCPs',
    font=dict(size=14),
    height=500,
    plot_bgcolor='white',
    margin=dict(l=80, r=40, t=80, b=40),
    bargap=0.1
)

fig4.show()
```




```python
# 4. Distribution of Suggestions per HCP
# Count suggestions per HCP
hcp_values = df['Account: Name'].value_counts()

# Create histogram bins
counts, bins = np.histogram(hcp_values, bins=30)
bin_centers = 0.5 * (bins[:-1] + bins[1:])

# Fancy labeled histogram
fig4 = go.Figure()
fig4.add_trace(go.Bar(
    x=bin_centers,
    y=counts,
    text=counts,
    textposition='outside',
    marker=dict(color='rgba(0, 128, 255, 0.7)',
                line=dict(color='rgba(0, 128, 255, 1.0)', width=1))
))
fig4.update_layout(
```

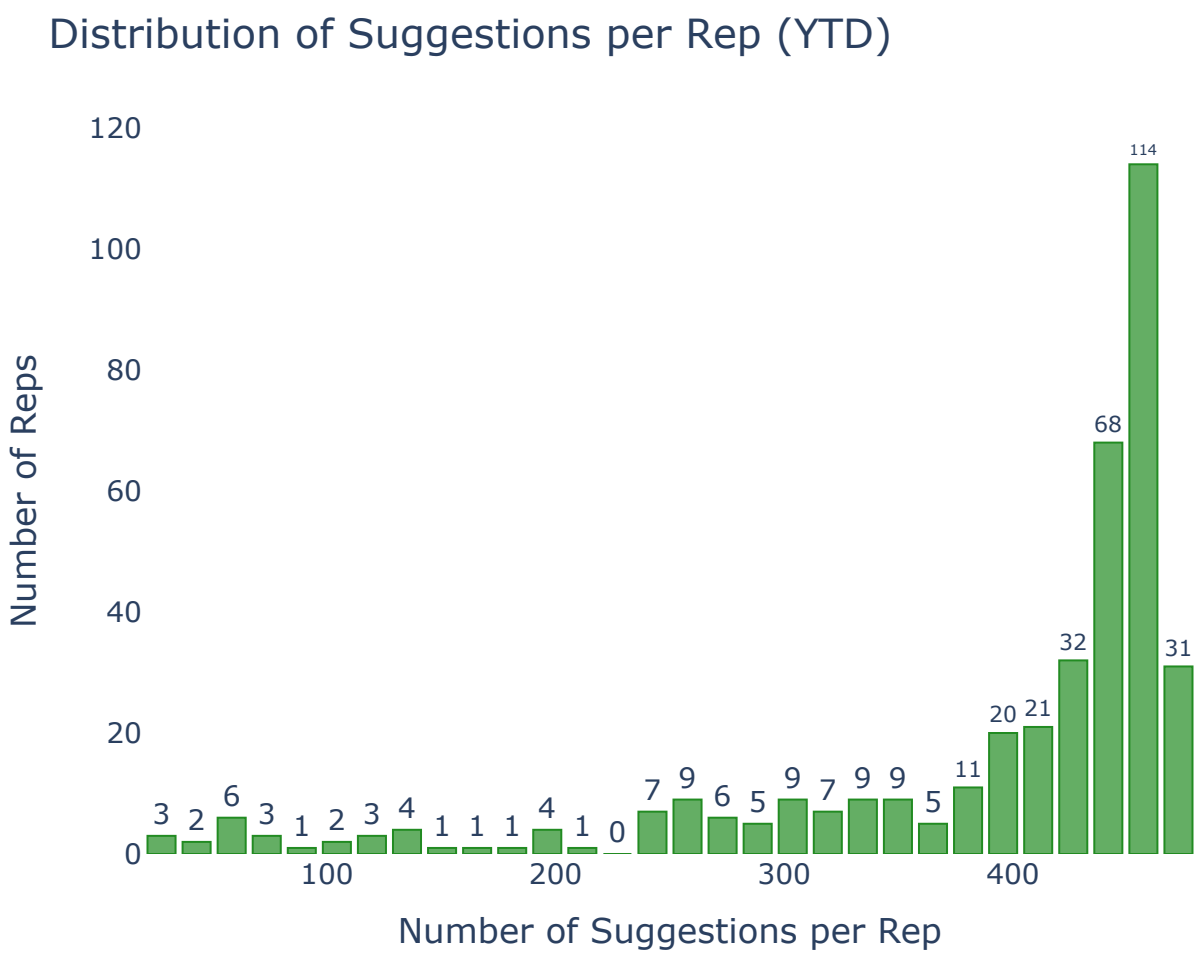
```
    title='Distribution of Suggestions per HCP (YTD)',
    xaxis_title='Number of Suggestions per HCP',
    yaxis_title='Number of HCPs',
    font=dict(size=14),
    height=500,
    plot_bgcolor='white',
    margin=dict(l=80, r=40, t=80, b=40)
)
fig4.show()

# Create readable summary table for ranges
bucket_edges = [0, 2, 5, 10, 15, float('inf')]
bucket_labels = ['1-2', '3-5', '6-10', '11-15', '>15']
bucket_counts = pd.cut(hcp_values, bins=bucket_edges, labels=bucket_labels, include_low
est=True).value_counts().sort_index()
percentages = (bucket_counts.values / len(hcp_values) * 100).round(1)

# Safely format the table
summary_df = pd.DataFrame({
    'Range of Suggestions per HCP': bucket_labels,
    'Number of HCPs': bucket_counts.values,
    '% of Total HCPs': [f"{p:.1f}%" for p in percentages]
})

summary_df
```



	Range of Suggestions per HCP	Number of HCPs	% of Total HCPs
0	1–2	16967	39.7%
1	3–5	17791	41.6%
2	6–10	7321	17.1%
3	11–15	622	1.5%
4	>15	35	0.1%

```
# 5. Distribution of Suggestions per Rep

# Get count of suggestions per Rep
rep_values = df['Owner: Full Name'].value_counts()

# Create histogram bins
rep_counts, rep_bins = np.histogram(rep_values, bins=30)
rep_bin_centers = 0.5 * (rep_bins[:-1] + rep_bins[1:])

# Create labeled histogram-style bar chart
fig5 = go.Figure()

fig5.add_trace(go.Bar(
    x=rep_bin_centers,
    y=rep_counts,
    text=rep_counts,
    textposition='outside',
    marker=dict(color='rgba(34, 139, 34, 0.7)',  # green tone
                line=dict(color='rgba(34, 139, 34, 1.0)', width=1))
))

fig5.update_layout(
    title='Distribution of Suggestions per Rep (YTD)',
    xaxis_title='Number of Suggestions per Rep',
    yaxis_title='Number of Reps',
    font=dict(size=14),
    height=500,
    plot_bgcolor='white',
    margin=dict(l=80, r=40, t=80, b=40)
)

fig5.show()

# ----------------------------
# Summary Table Below the Chart
# ----------------------------

# Define custom buckets
bucket_edges = [0, 50, 100, 200, 300, float('inf')]
bucket_labels = ['1-50', '51-100', '101-200', '201-300', '>300']

# Bin reps by number of suggestions
bucket_counts = pd.cut(rep_values, bins=bucket_edges, labels=bucket_labels, include_low
```

```
est=True).value_counts().sort_index()
percentages = (bucket_counts.values / len(rep_values) * 100).round(1)

# Format table
rep_summary_df = pd.DataFrame({
    'Range of Suggestions per Rep': bucket_labels,
    'Number of Reps': bucket_counts.values,
    '% of Total Reps': [f"{p:.1f}%" for p in percentages]
})

# Display table
rep_summary_df
```



	Range of Suggestions per Rep	Number of Reps	% of Total Reps
0	1–50	5	1.3%
1	51–100	11	2.8%
2	101–200	12	3.0%
3	201–300	33	8.4%
4	>300	334	84.6%